



**International Cybersecurity and Digital Forensics Academy
(ICDFA)**

SBT-DF202 — Computer and Digital Forensics

Lab 3: Data Carving with XXD, Binwalk and Scalpel

Finding, reading, and rebuilding files that the file system can no longer see

Student Name: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392

Instructor: Dr. Aminu Idris

*(CCNA, CompTIA Security+, CEH, OSCP, CISSP | MPCSEAN —
Cybersecurity Educator & Mentor)*

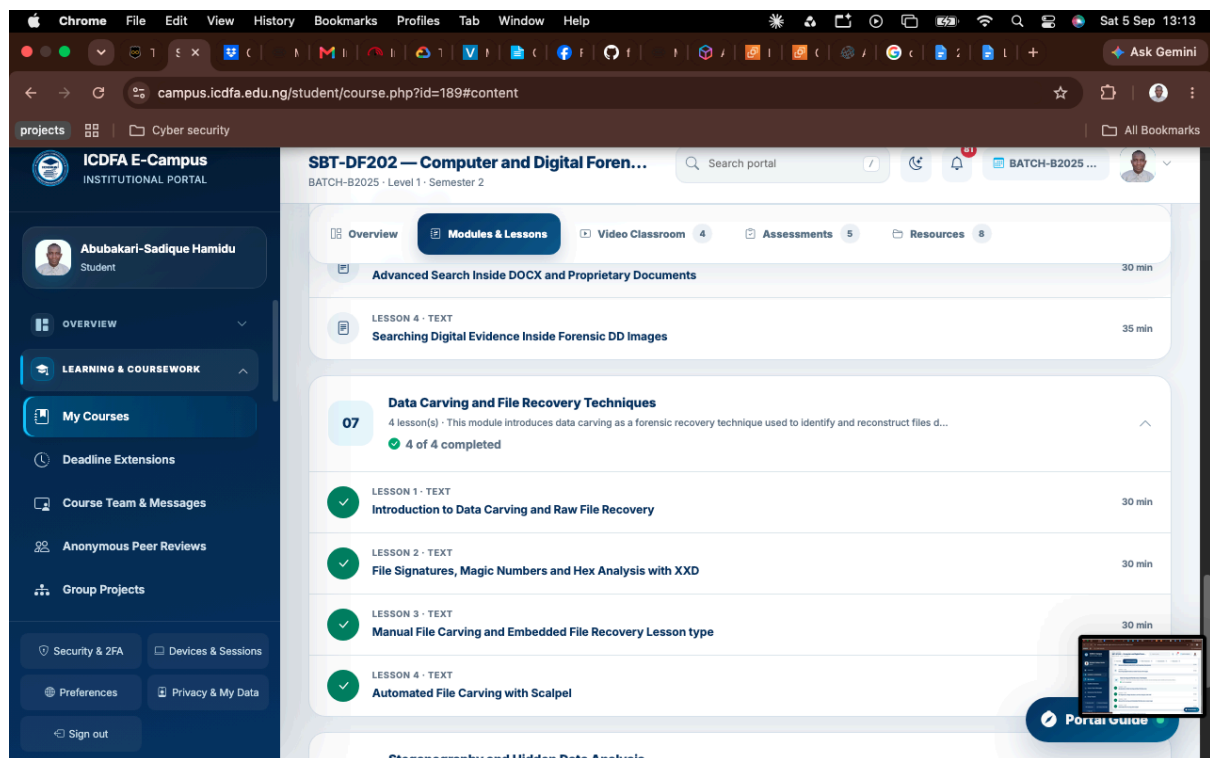
Platform Used: Ubuntu Server 22.04 (AWS EC2 instance)

Submission Date: 05 September 2026

1. Before I Started

1.1 Reviewing Module 7

Before touching any commands, I went through all four Module 7 lessons — data carving fundamentals, file signatures and hex analysis with xxd, manual carving with binwalk and dd, and automated carving with Scalpel. The lab instructions say this material is a detailed walkthrough of what's expected here, and that turned out to be true, so it was worth doing properly rather than skimming it.



1.2 Getting the Evidence Files

The lab page gave me a working download link for one of the three evidence files — Ch01InChap01.dd, hosted on Dropbox. The other two, J_ub_law.jpg and 120M.7z, were just named in the instructions with no actual link attached. I checked the lab page itself, the lesson pages for Module 7, and looked for any kind of resources or attachments section, but never found them anywhere on the IC DFA portal.

Since I couldn't source them from the course, I got both files from a public GitHub repository that several digital forensics courses use as a source for these exact training files:

<https://github.com/frankwxu/Digital-Forensics-Basic-Book>

The filenames matched exactly what the lab asked for, so I'm confident these are the intended files, just reached through a different route than the course portal. The Ch01InChap01.dd file came from the Dropbox link the lab actually provided.

1.3 A Note on Platform

The lab asks for Kali Linux or an Ubuntu forensic workstation/VM. I started on my own Mac, but hit a wall almost immediately — there's no Homebrew package for Scalpel at all, and even something as basic as md5sum works differently on macOS (it's md5 -r there

instead). Rather than patch together substitutes for half the lab, I spun up a plain Ubuntu Server 22.04 instance on AWS and did all the actual analysis there over SSH, working from my Mac's terminal.

Every tool the lab needs — The Sleuth Kit, Binwalk, Scalpel, p7zip — installed cleanly through Ubuntu's own package manager on that instance, so in the end this matched the lab's required platform exactly, just running in the cloud rather than on a local VM. Evidence files were copied up to the instance with scp, and once all the analysis was done, everything was copied back down to my Mac for writing this report, with hashes re-checked after each transfer to make sure nothing got altered along the way.

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~ — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35

ubuntu@ip-172-31-42-19:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo

ubuntu@ip-172-31-42-19:~$ sudo apt install -y sleuthkit binwalk scalpel p7zip-full xxd
Note, selecting '7zip' instead of 'p7zip-full'
sleuthkit is already the newest version (4.12.1+dfsg-3build1).
xxd is already the newest version (2:9.1.2141-1ubuntu4.8).
xxd set to manually installed.
Installing:
 7zip binwalk scalpel

Installing dependencies:
adwaita-icon-theme  libgles2  libqt6network6  libxrandr2
arj                 libglu1-mesa  libqt6opengl6  libxrender1
at-spi2-common     libglut3.12  libqt6openglwidgets6  libxres1
at-spi2-core       libglvnd0    libqt6printsupport6  libxshmfence1
bubblewrap         libglx-mesa0  libqt6qml6  libxss1
cpp                libglx0      libqt6qmlmeta6  libxtst6
cpp-15            libglycin-2-0  libqt6qmlmodels6  libxxf86vm1
cpp-15-x86-64-linux-gnu  libgomp1  libqt6qmlworkerscript6  libzopfli1
cpp-x86-64-linux-gnu  libgraphite2-3  libqt6quick6  linux-libc-dev
cramfsswap        libgtk-3-0t64  libqt6sql6  manpages-dev
dconf-gsettings-backend  libgtk-3-bin  libqt6sql6-sqlite  mesa-libgallium
dconf-service     libgtk-3-common  libqt6svg6  mesa-vulkan-drivers

Setting up libqt6printsupport6:amd64 (6.10.2+dfsg-7) ...
Setting up libqt5designer5:amd64 (5.15.18-1) ...
Setting up python3-pyqt6 (6.10.2-2build5) ...
Setting up libqt6waylandcompositor6:amd64 (6.10.2-4) ...
Setting up pyqt6-dev-tools (6.10.2-2build5) ...
Setting up qtwayland5:amd64 (5.15.18-1) ...
Setting up libgdk-pixbuf-2.0-0:amd64 (2.44.5+dfsg-4ubuntu1) ...
Setting up qt6-wayland:amd64 (6.10.2-4) ...
Setting up python3-pyqt5 (5.15.11+dfsg-3build3) ...
Setting up gtk-update-icon-cache (4.22.4+ds-0ubuntu0.1) ...
Setting up python3-pyqt5.qtopengl (5.15.11+dfsg-3build3) ...
Setting up python3-pyqtgraph (0.14.0-5) ...
Setting up adwaita-icon-theme (50.0-1) ...
update-alternatives: using /usr/share/icons/Adwaita/cursor.theme to provide /usr/share/icons/default/index.theme (x-cursor-th
eme) in auto mode
Setting up libgtk-3-0t64:amd64 (3.24.52-0ubuntu1) ...
Setting up librsvg2-common:amd64 (2.61.3+dfsg-3) ...
Setting up libgtk-3-bin (3.24.52-0ubuntu1) ...
Setting up qt5-gtk-platformtheme:amd64 (5.15.18+dfsg-1ubuntu1) ...
Setting up qt6-gtk-platformtheme:amd64 (6.10.2+dfsg-7) ...
Processing triggers for libc-bin (2.43-2ubuntu2.3) ...
Processing triggers for libgdk-pixbuf-2.0-0:amd64 (2.44.5+dfsg-4ubuntu1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-42-19:~$
```

2. Scenario and Objectives

2.1 Scenario

The scenario for this lab: a forensic storage image contains files that may no longer show up through normal file browsing, and some of the file metadata may be missing, damaged, or just not trustworthy. Working as a junior analyst, the job is to look at the raw content of the evidence directly, recognize files by their signatures rather than their names, pull out anything

embedded inside other files, and recover deleted files from a USB image — documenting every step along the way so someone else could repeat it.

2.2 What This Lab Was Meant to Cover

- Why data carving matters once file-system metadata can't be trusted.
- Using xxd to look at binary file content and spot headers and footers by eye.
- Recognizing the JPEG SOI and EOI signatures specifically.
- Rebuilding a binary file from a hex dump.
- Using The Sleuth Kit to look at a file system down to the inode, block, and sector level.
- Finding embedded content with Binwalk.
- Recovering deleted files from a USB image with Scalpel.
- Checking recovered evidence against hashes.
- Writing all of this up properly — commands, screenshots, findings, and the parts that didn't go perfectly.

3. Setting Up

I made a working directory on the Ubuntu instance (~/.lab3-data-carving/) with a separate folder for each part of the lab, plus one for the original evidence and one just for hash logs, so nothing from my own output could ever get mixed in with the untouched originals. All three evidence files went into evidence/ and got hashed immediately, before I ran a single analysis command against them, so I'd have a clean baseline to check everything against afterward.

```
$ ls -lh
$ md5sum * > ../hashes/original_hashes_md5.txt
$ sha256sum * > ../hashes/original_hashes_sha256.txt
```

```

Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~$ mkdir -p ~/lab3-data-carving/{evidence,hashes,partA_hex_analysis,partB_metadata,partC_tsk,partD_bin,walk,partE_usb_prep,partF_scalpel,screenshots,report}
ubuntu@ip-172-31-42-19:~$ cd ~/lab3-data-carving/evidence
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ scp -i ~/Downloads/sadique.pem ~/Downloads/Ch01InChap01.dd ~/Downloads/Jubub_law.jpg ~/Downloads/120M.7z ubuntu@172-31-42-19:~/lab3-data-carving/evidence/
-bash: 172-31-42-19: No such file or directory
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$

```

3.1 Original Evidence — Hash Baseline

File	Size	MD5	SHA-256 (shortened)
Ch01InChap01.dd	1.5M	a117773bcf1fc88ec0ab8e0a349fbbcb	3ce8053e4f3d9c8ab98b3aad...a7b122
J_ub_law.jpg	1.7M	83a360ac7f7e0ca318e5bfe39f95f137	238ff34393c50e52c0e8b14f...72a91468
120M.7z	36M	dfe7b5424e54cd1bf50d5df47aceeb3c	2d2a3d93c9ec65bcad9f89c9...6ab6a58ce

3.2 Checking the Tools Were Actually There

Before starting Part A, I confirmed every tool the lab needs was actually installed, so I wasn't halfway through a step later and had to stop and install something:

```
$ which mmls fsstat fls icat blkcat img_stat binwalk scalpel 7z xxd
/usr/bin/mmls
/usr/bin/fsstat
/usr/bin/fls
/usr/bin/icat
/usr/bin/blkcat
/usr/bin/img_stat
/usr/bin/binwalk
/usr/bin/scalpel
/usr/bin/7z
/usr/bin/xxd
```

4. Part A — File Signatures and Hex Analysis

The goal here was to prove J_ub_law.jpg is really a JPEG by looking at its actual bytes rather than trusting the file extension, find exactly where the header and footer sit, dump the whole file to hex, rebuild it from that hex dump, and check the rebuilt copy against the original with hashes.

4.1 Finding the Header and Footer

```
$ xxd J_ub_law.jpg | head
$ xxd J_ub_law.jpg | tail
```

Why I did this: a file extension can say anything — it doesn't prove what a file actually is. Looking at the raw bytes, the file starts with `ffd8`, which is the JPEG SOI (Start of Image) marker, and ends with `ffd9`, the EOI (End of Image) marker. Those are the same two markers Scalpel will later be told to search for automatically in Part F, so it helped to see them by hand first.

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35

ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd J_ub_law.jpg | head
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....
00000050: 0000 1001 0200 0900 0000 e600 0000 1201 .....
00000060: 0300 0100 0000 0100 0000 1501 0300 0100 .....
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00 .....
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801 .....(
00000090: 0300 0100 0000 0200 0000 3101 0200 2100 .....1...!
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd J_ub_law.jpg | tail
0019d160: ecfe 3d52 1918 3008 da88 b489 7f22 feae ..=R..0.....".
0019d170: 2def 629c 7ab5 16bc 7acd 5750 6ba9 63a7 -.b.z...z.WPk.c.
0019d180: 9155 0524 f1a8 648f 4d6a f9c7 95e4 8981 .U.$..d.Mj.....
0019d190: 55d6 1519 8293 6360 41e0 fbd5 0530 3aa9 U.....c`A....0:.
0019d1a0: 0bab 554f 4d91 346a 25a7 99d9 2a52 a1bd ..UQM.4j%...*R..
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80 )O.J.Y..(.QZ.v:.
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45 ....y....S..._E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1 *.-$....i.y....
0019d1e0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565 ..RT.8...o..Z.T.e
0019d1f0: d351 9ebf ffd9 ..Q....
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$
```

4.2 Plain Hex Dump and Searching for the Signatures

```
$ xxd -p J_ub_law.jpg > J_ub_law_plain_hex.txt
$ xxd -p J_ub_law.jpg | grep -i "ffd8"
$ xxd -p J_ub_law.jpg | tail | grep -i "ffd9"
```

***Why I did this:** the `-p` flag strips out the offset column and the ASCII sidebar, leaving just a continuous string of hex. That makes it possible to `grep` for a signature reliably, and it's also the exact format needed to reverse the dump back into a real file in the next step.*

```

Terminal Shell Edit View Window Help
Downloads - ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35

00000050: 0000 1001 0200 0900 0000 e600 0000 1201 .....
00000060: 0300 0100 0000 0100 0000 1501 0300 0100 .....
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00 .....
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801 .....
00000090: 0300 0100 0000 0200 0000 3101 0200 2100 .....1...!
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd J_ub_law.jpg | tail
0019d160: ecfe 3d52 1918 3008 da88 b489 7f22 feae ..=R..0....."
0019d170: 2def 629c 7ab5 16bc 7acd 5750 6ba9 63a7 -.b.z...z.WPk.c.
0019d180: 9155 0524 f1a8 648f 4d6a f9c7 95e4 8981 .U$.d.Mj.....
0019d190: 55d6 1519 8293 6360 41e0 fbd5 0530 3aa9 U....c`A...0:.
0019d1a0: 0bab 554f 4d91 346a 25a7 99d9 2a52 a1bd ..UOM.4j%...*R..
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80 )O.J.Y..(.QZ.v:.
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45 .....Y....S..._E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1 *.-$....i.y....
0019d1e0: fff0 5254 0938 040e 6ff5 f75a 9a54 f565 ..RT.8...o..Z.T.e
0019d1f0: d351 9ebf ffd9 .Q....
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg > ../partA_hex_analysis/J_ub_law_plain_hex.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ffd8ffe117f0457869660000492a000800000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000601
030001000000020000000f01020012000000d40000001001020007000000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f7000000280103000100
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | grep -i "ffd8"
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ffd8ffe117f0457869660000492a000800000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000601
030001000000020000000f01020012000000d40000001001020007000000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f7000000280103000100
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | tail | grep -i "ffd9"
ad6994790789d1e1ff0052540938040e6ff5f75a9a54f565d3519ebfffd9
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | tail | grep -i "ffd8"
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$

```

4.3 Rebuilding the File and Checking It Matches

```

$ xxd -r -p J_ub_law_plain_hex.txt J_ub_law_reconstructed.jpg
$ md5sum J_ub_law.jpg J_ub_law_reconstructed.jpg
$ sha256sum J_ub_law.jpg J_ub_law_reconstructed.jpg
$ file J_ub_law.jpg J_ub_law_reconstructed.jpg

```

Why I did this: this is really the whole point of the exercise — a hex dump holds every single byte of a file, so reversing it should give back exactly the same file, byte for byte. The MD5 and SHA-256 hashes came back identical for both files, and running file on both also showed the exact same JPEG/Exif details (same camera info, same dimensions), so I had three separate confirmations rather than just one.


```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35

ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ ls -lh
total 39M
-rw-r--r-- 1 ubuntu ubuntu 36M Sep  5 07:43 120M.7z
-rw-r--r-- 1 ubuntu ubuntu 1.5M Sep  5 07:39 Ch01InChap01.dd
-rw-r--r-- 1 ubuntu ubuntu 1.7M Sep  5 07:40 J_ub_law.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ md5sum * > ../hashes/original_hashes_md5.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ sha256sum * > ../hashes/original_hashes_sha256.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ cat ../hashes/original_hashes_md5.txt ../hashes/original_hashes_sha256.txt
dfe7b5424e54cd1bf50d5df47aceeb3c 120M.7z
a117773bcf1fc88ec0ab8e0a349fbbcb Ch01InChap01.dd
83a360ac7f7e0ca318e5bfe39f95f137 J_ub_law.jpg
2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce 120M.7z
3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122 Ch01InChap01.dd
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 J_ub_law.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$
```

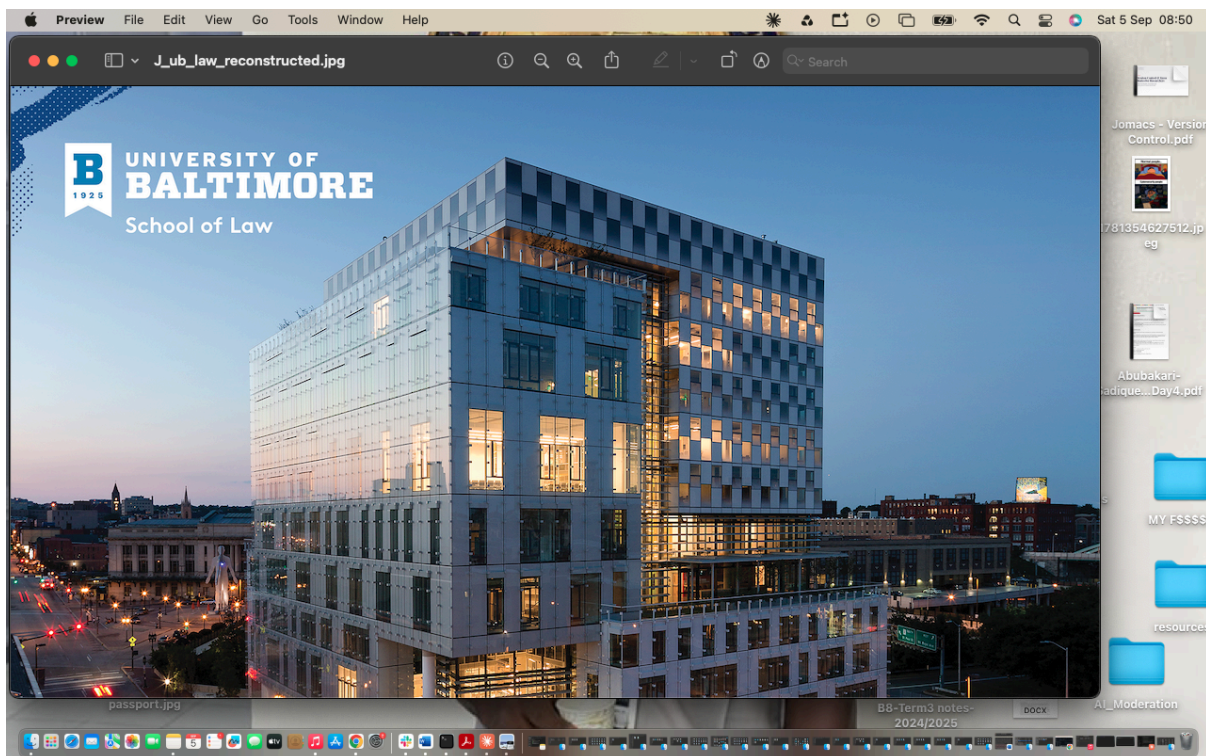
```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35

0019d170: 2def 629c 7ab5 16bc 7acd 5750 6ba9 63a7 -.b.z...z.WPk.c.
0019d180: 9155 0524 f1a8 648f 4d6a f9c7 95e4 8981 .U$.d.Mj.....
0019d190: 55d6 1519 8293 6360 41a0 fbd5 0530 3aa9 U....c'A....0:.
0019d1a0: 0bab 554f 4d91 346a 25a7 99d9 2a52 a1bd ..UOM.4j%...*R..
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80 )O.J.Y..(.QZ.v:.
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45 .....y....S....E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1 *.-$.....i.y....
0019d1e0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565 ..RT.8..o..Z.T.e
0019d1f0: d351 9ebf ffd9 .Q....
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg > ../partA_hex_analysis/J_ub_law_plain_hex.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ffd8ffe117f04578696000049492a00080000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000601
03000100000020000000f01020012000000d40000001001020009000000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f7000000280103000100ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.
jpg | grep -i "ffd8"
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ffd8ffe117f04578696000049492a00080000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000601
03000100000020000000f01020012000000d40000001001020009000000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f7000000280103000100ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../p
artA_hex_analysis/J_ub_law_plain_hex.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | tail | grep -i "ffd9"
ad6994790789d1e1ff0052540938040e6ff5f75a9a54f565d3519ebfffd9
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | tail | grep -i "ffd8"
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -r -p ../partA_hex_analysis/J_ub_law_plain_hex.txt ../partA_hex_anal
ysis/J_ub_law_reconstructed.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ md5sum J_ub_law.jpg ../partA_hex_analysis/J_ub_law_reconstructed.jpg
83a360ac7f7e0ca318e5bfe39f95f137 J_ub_law.jpg
83a360ac7f7e0ca318e5bfe39f95f137 ../partA_hex_analysis/J_ub_law_reconstructed.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ sha256sum J_ub_law.jpg ../partA_hex_analysis/J_ub_law_reconstructed.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 J_ub_law.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 ../partA_hex_analysis/J_ub_law_reconstructed.jpg
```

```
Terminal Shell Edit View Window Help
Downloads - ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35

0019d170: 2def 629c 7ab5 16bc 7acd 5750 6ba9 63a7 -.b.z...z.WPk.c.
0019d180: 9155 0524 f1a8 648f 4d6a f9c7 95e4 8981 .U$.d.Mj.....
0019d190: 55d6 1519 8293 6360 41e0 fbd5 0530 3aa9 U.....'A....0:.
0019dia0: 0bab 554f 4d91 346a 25a7 99d9 2a52 a1bd ..UOM.4j%...*R..
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80 )O.J.Y..(.QZ.v:..
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45 ....y...S..._E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1 .*-$....i.y....
0019die0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565 ..RT.8..o..Z.T.e
0019d1f0: d351 9ebf ffd9 .Q....

ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg > ../partA_hex_analysis/J_ub_law_plain_hex.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ffd8ffe117f045786966000049492a00080000010000001030001000000
401300000101030001000000d00c00000201030003000000ce000000601
030001000000020000000f01020012000000d4000000100102000900000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f7000000280103000100ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.
jpg | grep -i "ffd8"
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../partA_hex_analysis/J_ub_law_plain_hex.txt
ffd8ffe117f045786966000049492a00080000010000001030001000000
401300000101030001000000d00c00000201030003000000ce000000601
030001000000020000000f01020012000000d4000000100102000900000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f7000000280103000100ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ head -c 300 ../p
artA_hex_analysis/J_ub_law_plain_hex.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | tail | grep -i "ffd9"
ad6994790789d1e1ff0052540938040e6ff5f75a9a54f565d3519ebfffd9
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -p J_ub_law.jpg | tail | grep -i "ffd8"
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ xxd -r -p ../partA_hex_analysis/J_ub_law_plain_hex.txt ../partA_hex_anal
ysis/J_ub_law_reconstructed.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ md5sum J_ub_law.jpg ../partA_hex_analysis/J_ub_law_reconstructed.jpg
83a360ac7f7e0ca318e5bfe39f95f137 J_ub_law.jpg
83a360ac7f7e0ca318e5bfe39f95f137 ../partA_hex_analysis/J_ub_law_reconstructed.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ sha256sum J_ub_law.jpg ../partA_hex_analysis/J_ub_law_reconstructed.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 J_ub_law.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 ../partA_hex_analysis/J_ub_law_reconstructed.jpg
```



4.4 What This Showed

The rebuilt file matched the original three different ways — MD5, SHA-256, and file's structural read-out (right down to the full Exif metadata) — so a hex dump really is a lossless copy of a file's content. That's the whole basis for why carving works at all: if you can find the right bytes, you can rebuild the file, with or without the original file system's help.

5. Part B — Looking for Metadata

This part was about checking J_ub_law.jpg for anything that looked like camera info, editing software, a name, or a date, using strings and xxd — and being honest about writing "not found" for anything that genuinely wasn't there instead of guessing.

```
$ strings J_ub_law.jpg > J_ub_law_strings.txt
$ grep -i -E
"canon|nikon|sony|adobe|photoshop|gimp|exif|software|camera|date|author|copyright"
J_ub_law_strings.txt
$ xxd J_ub_law.jpg | head -50
$ grep -i "gps|location|latitude|longitude" J_ub_law_strings.txt
```

Why I did this: *strings* pulls out any readable text hiding inside otherwise unreadable binary data — camera and software names are often stored as plain ASCII inside a JPEG's Exif or XMP sections. *xxd* let me double-check the raw bytes around anything strings found, and I ran a separate GPS-specific search afterward rather than just assuming location data was absent — better to actually check than guess.

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ strings evidence/J_ub_law.jpg > partB_metadata/J_ub_law_strings.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ cat partB_metadata/J_ub_law_strings.txt
Exif
NIKON CORPORATION
NIKON D4
Adobe Photoshop 21.1 (Macintosh)
2020:08:20 10:20:05
HOWARD KORN
0221
2013:10:08 18:56:21
2013:10:08 18:56:21
2027750
17.0-35.0 mm f/2.8
Adobe_CM
Adobe
b34r
7GWgw
AQaq"
dEU6te
'7GWgw
2VtWX
I;[h
[$Wik
S_b[
Suhf
kC{ LO
uq\j`
Y{h}
{^v>
04YC[hakK
X-QI:
!2Y:$C
```

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ grep -i -E "canon|nikon|sony|adobe|photoshop|gimp|exif|software|camera|date|author|copyright" partB_metadata/J_ub_law_strings.txt
Exif
NIKON CORPORATION
NIKON D4
Adobe Photoshop 21.1 (Macintosh)
Adobe_CM
Adobe
"Photoshop 3.0
Adobe_CM
Adobe
http://ns.adobe.com/xap/1.0/
" id="W5M0MpCehiHzreSzNTczkc9d"?> <x:xmpmeta xmlns:x="adobe:ns:meta/" x:xmptk="Adobe XMP Core 6.0-c002 79.164352,
2020/01/30-15:50:38 "> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"> <rdf:Description r
df:about="" xmlns:aux="http://ns.adobe.com/exif/1.0/aux/" xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:photos
hop="http://ns.adobe.com/photoshop/1.0/" xmlns:xmp="http://ns.adobe.com/xap/1.0/" xmlns:xmpMM="http://ns.adobe.com
/xap/1.0/mm/" xmlns:stRef="http://ns.adobe.com/xap/1.0/sType/ResourceRef#" xmlns:stEvt="http://ns.adobe.com/xap/1.
0/sType/ResourceEvent#" xmlns:crs="http://ns.adobe.com/camera-raw-settings/1.0/" xmlns:lr="http://ns.adobe.com/lig
htroom/1.0/" aux:ApproximateFocusDistance="4294967295/1" aux:ImageNumber="115422" aux:Lens="17.0-35.0 mm f/2.8" au
x:LensID="99" aux:LensInfo="170/10 350/10 28/10 28/10" aux:SerialNumber="2027750" dc:fo
:DateCreated="2013-10-08" photoshop:LegacyIPTCDigest="5E379AE71DB032222FA2DE651E47F460"
otoshop:ICCPProfile="Adobe RGB (1998)" xmp:CreateDate="2013-10-08T18:56:21" xmp:CreatorT
p:Label="Approved" xmp:MetadataDate="2020-08-20T10:20:05-04:00" xmp:ModifyDate="2020-08-
ing="0" xmpMM:DocumentID="adobe:docid:photoshop:307cd9db-3a87-6148-9874-c6730fd42952" x
7f6b27-04e9-43e7-ae79-c3f2fb9c2ac9" xmpMM:OriginalDocumentID="B04B02E367C62737F8E786497
="True" crs:AutoLateralCA="0" crs:AutoWhiteVersion="134348800" crs:Blacks2012="0" crs:B
on="0" crs:CameraProfile="Embedded" crs:CameraProfileDigest="54650A341B5B5CCAE8442D0B43/
crs:ColorNoiseReduction="0" crs:Contrast2012="0" crs:ConvertToGrayscale="False" crs:Def
efringeGreenHueHi="60" crs:DefringeGreenHueLo="40" crs:DefringePurpleAmount="0" crs:Def
efringePurpleHueLo="30" crs:Exposure2012="0.00" crs:GrainAmount="0" crs:GreenHue="0" crs
asCrop="False" crs:HasSettings="True" crs:Highlights2012="0" crs:HueAdjustmentAqua="0" c
rs:HueAdjustmentGreen="0" crs:HueAdjustmentMagenta="0" crs:HueAdjustmentOrange="0" crs:
```



```

ubuntu@ip-172-31-42-19:~/lab3-data-carving$ xxd evidence/J_ub_law.jpg | head -50
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*
00000010: 0000 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....
00000050: 0000 1001 0200 0900 0000 e600 0000 1201 .....
00000060: 0300 0100 0000 0100 0000 1501 0300 0100 .....
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00 .....
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801 .....(
00000090: 0300 0100 0000 0200 0000 3101 0200 2100 .....1...!
000000a0: 0000 ff00 0000 3201 0200 1400 0000 2001 .....2.....
000000b0: 0000 9882 0200 0c00 0000 3401 0000 9e9c .....4.....
000000c0: 0100 5200 0000 4001 0000 6987 0400 0100 ..R...@...i....
000000d0: 0000 9401 0000 4404 0000 0800 0800 0800 .....D.....
000000e0: 4e49 4b4f 4e20 434f 5250 4f52 4154 494f NIKON CORPORATIO
000000f0: 4e00 4e49 4b4f 4e20 4434 00c0 c62d 0010 N.NIKON D4...-
00000100: 2700 00c0 c62d 0010 2700 0041 646f 6265 '....-...'Adobe
00000110: 2050 686f 746f 7368 6f70 2032 312e 3120 Photoshop 21.1
00000120: 284d 6163 696e 746f 7368 2900 3230 3230 (Macintosh).2020
00000130: 3a30 383a 3230 2031 303a 3230 3a30 3500 :08:20 10:20:05.
00000140: 484f 5741 5244 204b 4f52 4e00 3200 3000 HOWARD KORN.2.0.
00000150: 3100 3300 2c00 2000 6300 6100 6d00 7000 1.3.,, .c.a.m.p.
00000160: 7500 7300 2c00 2000 4d00 6500 7200 7200 u.s.,, .M.e.r.r.
00000170: 6900 6300 6b00 2000 5300 6300 6800 6f00 i.c.k. .S.c.h.o.
00000180: 6f00 6c00 2000 6f00 6600 2000 4200 7500 o.l. .o.f. .B.u.
00000190: 7300 6900 6e00 6500 7300 7300 0000 0000 s.i.n.e.s.s....
000001a0: 2900 9a82 0500 0100 0000 8603 0000 9d82 ).....
000001b0: 0500 0100 0000 8e03 0000 2288 0300 0100 .....".
000001c0: 0000 0100 0000 2788 0300 0100 0000 c800 .....'.
000001d0: 0000 3088 0300 0100 0000 0200 0000 0090 ..0.....
000001e0: 0700 0400 0000 3032 3231 0390 0200 1400 .....0221.....

```

5.1 What I Found

Category	Result
Camera	Found — Manufacturer: NIKON CORPORATION, Model: NIKON D4. Shows up in file, in strings, and directly in the raw xxd bytes around offset 0x00E4–0x00FD.
Software	Found — Adobe Photoshop 21.1 (Macintosh), Adobe Photoshop CS6 (Macintosh), Adobe Photoshop Camera Raw 8.3 (Macintosh), and a plain reference to GIMP.
Name / Author	Found — "HOWARD KORN" shows up in the XMP rights field and in the raw Exif data. Separately, a "Copyright 1999 Adobe Systems Incorporated" string also appears, but that's boilerplate text baked into the Adobe RGB (1998) color profile — not the actual photographer's copyright — so I kept the two apart rather than treating them as the same thing.
Date	Found — original capture date 2013-10-08 (xmp:CreateDate, 18:56:21), and a last-modified date of 2020-08-20T10:20:05-04:00, which also shows up in the raw bytes in standard Exif format as "2020:08:20 10:20:05".
GPS / Location	Not found — the only "gps"-matching hits were unrelated Photoshop internal tag names (PgPenum, PgPs). No actual location data is embedded in this file.

5.2 A Few Other Things Worth Mentioning

- The file's original name before it was renamed to J_ub_law.jpg: UB_10-8-13_cam2_0117.jpg (from crs:RawFileName) — the naming and date line up with a University of Baltimore source.
- Camera body serial number: 2027750.
- Lens: 17.0–35.0 mm f/2.8.
- The tags "campus" and "Merrick School of Business" actually show up twice, stored two completely different ways — once as old-style Windows XP Unicode tags visible directly in the xxd bytes around offset 0x140–0x199, and again as plain UTF-8 text in the modern XMP block. Same information, stored redundantly by two different metadata schemes.
- The XMP edit history shows this photo bouncing between JPEG, TIFF, and PSD formats across several editing sessions from 2014 through 2020.
- One thing that didn't quite add up: the Exif data claims the photo was captured at 4928×3280, but the actual JPEG is only 1920×1080. Most likely it was resized or re-exported at some point and the old capture-size metadata just never got updated.

6. Part C — Looking at the Disk Image Directly with The Sleuth Kit

For this part I switched over to Ch01InChap01.dd — an actual disk image rather than a single file — and used The Sleuth Kit to look at it from the partition level all the way down to individual sectors, pulling real numbers from my own output rather than the example values in the lecture.

6.1 Image and Partition Layout

```
$ img_stat Ch01InChap01.dd
Image Type: raw | Size: 1,474,560 bytes | Sector size: 512

$ mmls Ch01InChap01.dd
(no output)
```

Why I did this: 1,474,560 bytes works out to exactly 2,880 sectors of 512 bytes, which is the exact size of a classic 3.5" 1.44MB floppy disk. Floppies don't have a partition table (MBR) — the file system just starts at sector 0 — so mmls coming back empty is actually the expected, correct result here, not a failure. That became clear later by contrast, since Part E's real USB image did show a proper partition table when I ran the same command.

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ img_stat evidence/Ch01InChap01.dd
IMAGE FILE INFORMATION
-----
Image Type: raw

Size in bytes: 1474560
Sector size: 512
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ mmls evidence/Ch01InChap01.dd
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ fsstat evidence/Ch01InChap01.dd
FILE SYSTEM INFORMATION
-----
File System Type: FAT12

OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: ?3??M|

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
```

```
$ fsstat Ch01InChap01.dd
File System Type: FAT12
Total Range: 0 – 2879
FAT 0: 1 – 9 | FAT 1: 10 – 18
Root Directory: 19 – 32 | Cluster Area: 33 – 2879
Sector Size: 512 | Cluster Size: 512
```

```

Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ fsstat evidence/Ch01InChap01.dd
FILE SYSTEM INFORMATION
-----
File System Type: FAT12

OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: ?3??M|

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
-----
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512

```

6.3 Listing Files — Including the Deleted Ones

```
$ fls Ch01InChap01.dd
```

Address	Filename	Status
5	Client Info.mdb	Existing
8	Billing Letter.doc	Deleted
11	confirmation.txt	Deleted
13	Income.xls	Existing
15	letter1.txt	Deleted
17	Regrets.doc	Deleted

Why I did this: *fls* flags a deleted directory entry with an asterisk. Four of the six files on this floppy turned out to be marked deleted, which lines up nicely with the "deleted file recovery" scenario the Module 7 lecture describes — and gave me real addresses to actually recover something from, instead of just reading about it.

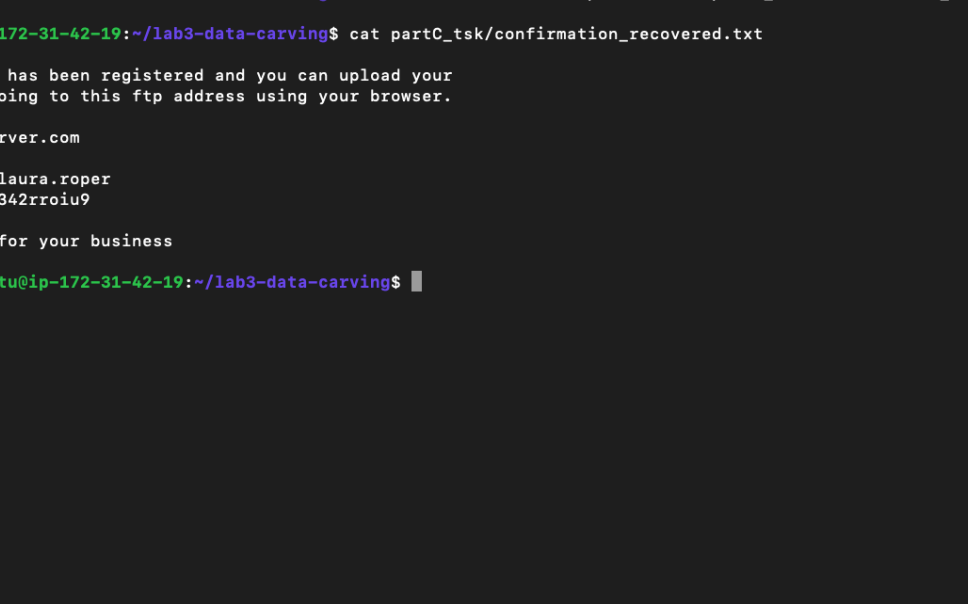
6.4 Pulling the Content Back Out (icat)

```
$ icat Ch01InChap01.dd 11 > confirmation_recovered.txt
$ icat Ch01InChap01.dd 8 > billing_letter_recovered.doc
```



```
$ icat Ch01InChap01.dd 15 > letter1_recovered.txt
$ icat Ch01InChap01.dd 17 > regrets_recovered.doc
```

Why I did this: *icat reads a file's content straight from its data units using its metadata address, completely skipping the normal check of whether the file shows up in a directory listing. That's exactly why this works on deleted files — deleting something under FAT just flips the directory entry to "reusable," it doesn't touch the actual data sitting in the clusters, or the pointer to where that data lives, until something else happens to overwrite it.*



The screenshot shows a macOS Terminal window with the following content:

```
Terminal  Shell  Edit  View  Window  Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ icat evidence/Ch01InChap01.dd 11 > partC_tsk/confirmation_recovered.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ cat partC_tsk/confirmation_recovered.txt
Laura,
The domain has been registered and you can upload your
files by going to this ftp address using your browser.

ftp.acmeserver.com

login id: laura.roper
password: 342rroi9

Thank you for your business

Georgeubuntu@ip-172-31-42-19:~/lab3-data-carving$
```

The terminal output shows the execution of the `icat` command to create a confirmation file, followed by the `cat` command to display its contents. The file contains a message from Laura, including a domain registration notice, an FTP address, login credentials, and a thank you note from George.

[illegible]

C:\Chap02\case files\in chapter\Regrets.doc, dated November 2003 — basically the same kind of hidden-metadata story as Part B, just showing up in a different file format this time.

6.5 Going One Level Deeper — istat and blkcat

```
$ istat Ch01InChap01.dd 11
```

```
Directory Entry: 11 | Not Allocated
```

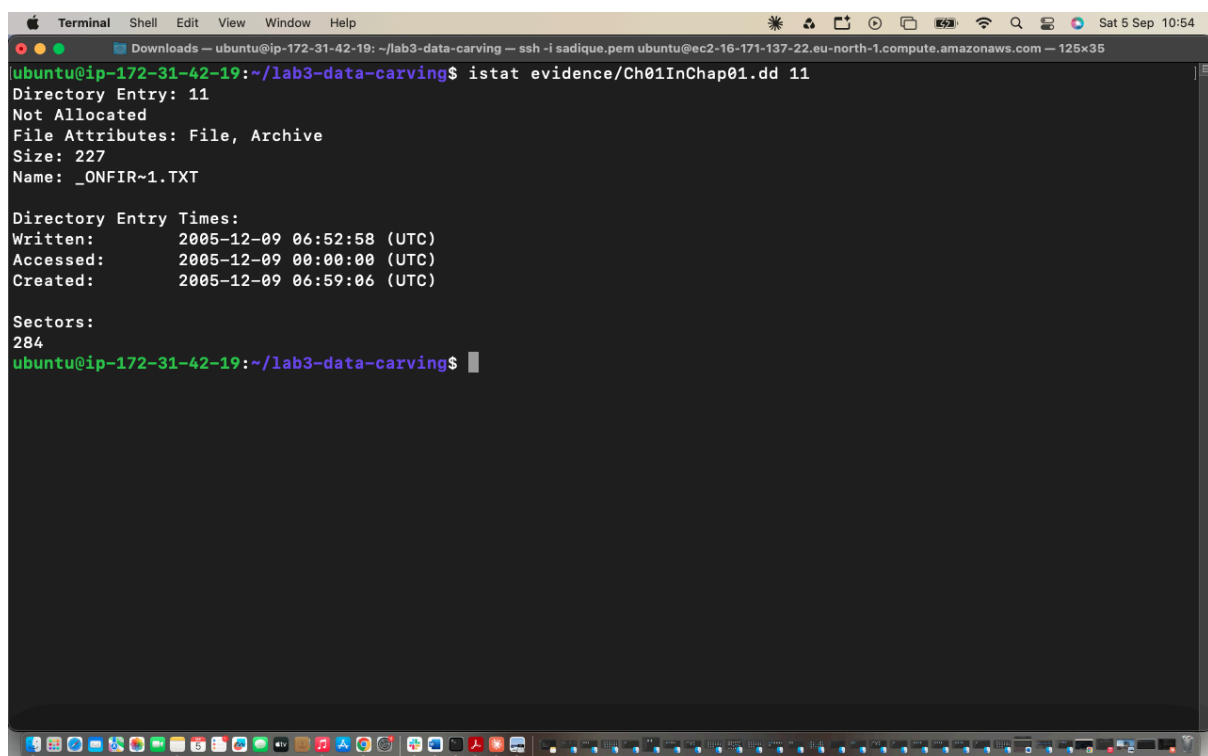
```
Name: _ONFIR~1.TXT | Size: 227
```

```
Sectors: 284
```

```
$ blkcat Ch01InChap01.dd 284
```

```
(raw sector content — matches the recovered confirmation.txt)
```

Why I did this: the recovered name comes back as "`_ONFIR~1.TXT`" instead of "`CONFIR~1.TXT`" — and that underscore isn't corruption, it's exactly how FAT marks a file deleted. It overwrites just the first byte of the filename with a special marker (0xE5), leaving everything else — the extension, the size, the timestamps, and most importantly the pointer to the data — completely untouched. That single-byte change is the whole reason recovery is possible at all. Running `blkcat` against the actual sector number (284) then reads the raw content directly, below even the file-metadata layer, and it matched what `icat` had already pulled out. So the full chain checks out: `fls` found the deleted name and address, `istat` gave the real sector number, and `blkcat` proved the bytes are genuinely still sitting there.



```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ istat evidence/Ch01InChap01.dd 11
Directory Entry: 11
Not Allocated
File Attributes: File, Archive
Size: 227
Name: _ONFIR~1.TXT

Directory Entry Times:
Written:      2005-12-09 06:52:58 (UTC)
Accessed:     2005-12-09 00:00:00 (UTC)
Created:      2005-12-09 06:59:06 (UTC)

Sectors:
284
ubuntu@ip-172-31-42-19:~/lab3-data-carving$
```

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 125x35
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ istat evidence/Ch01InChap01.dd 11
Directory Entry: 11
Not Allocated
File Attributes: File, Archive
Size: 227
Name: _ONFIR~1.TXT

Directory Entry Times:
Written:      2005-12-09 06:52:58 (UTC)
Accessed:     2005-12-09 00:00:00 (UTC)
Created:      2005-12-09 06:59:06 (UTC)

Sectors:
284
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ blkcat evidence/Ch01InChap01.dd 284
Laura,
The domain has been registered and you can upload your
files by going to this ftp address using your browser.

ftp.acmeserver.com

login id: laura.roper
password: 342rroui9

Thank you for your business

Georgeubuntu@ip-172-31-42-19:~/lab3-data-carving$
```

One small distinction I noted: `icat` gives back exactly the file's logical size (227 bytes, per the directory entry), while `blkcat` gives back the whole 512-byte sector, which can include leftover slack space past the end of the actual file. In this case the two lined up cleanly with nothing extra visible.

7. Part D — Looking for Embedded Files with Binwalk

`File_carving.docx`, the file the lab's demonstrated manual/automatic extraction walkthrough is built around, wasn't part of the evidence I had access to. The lab instructions actually plan for this — they say to document it as a limitation and run the Binwalk practice against whatever evidence is available instead, so that's what I did, using `J_ub_law.jpg` and `Ch01InChap01.dd`.

7.1 Running Binwalk

```
$ binwalk J_ub_law.jpg
0    0x0    JPEG image data, EXIF standard
12   0xC    TIFF image data, little-endian, offset of first IFD: 8
26844 0x68DC Copyright string: "Copyright 1999 Adobe Systems Incorporated"

$ binwalk Ch01InChap01.dd
(no signatures matched)
```

Why I did this: *Binwalk looks for known file signatures at any offset in a file, which is how it can spot something genuinely hidden inside another file. In this case though, all three hits inside `J_ub_law.jpg` are just parts of the JPEG's own normal Exif/XMP metadata block — not a separate embedded file. So the honest result here is that there's nothing actually embedded in either evidence file. That's still a real finding worth reporting, not a failed step.*

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ binwalk evidence/J_ub_law.jpg

DECIMAL      HEXADECEMAL  DESCRIPTION
-----
0            0x0          JPEG image data, EXIF standard
12           0xC          TIFF image data, little-endian offset of first image directory: 8
26844       0x68DC       Copyright string: "Copyright 1999 Adobe Systems Incorporated"

ubuntu@ip-172-31-42-19:~/lab3-data-carving$ binwalk evidence/Ch01InChap01.dd

DECIMAL      HEXADECEMAL  DESCRIPTION
-----

ubuntu@ip-172-31-42-19:~/lab3-data-carving$ binwalk -e evidence/J_ub_law.jpg

DECIMAL      HEXADECEMAL  DESCRIPTION
-----

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

ubuntu@ip-172-31-42-19:~/lab3-data-carving$ ls -la _J_ub_law.jpg.extracted/ 2>/dev/null
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ cd evidence
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$
```

7.2 Trying Extraction — and an Odd Inconsistency

```
$ binwalk -e J_ub_law.jpg
(empty result table)
WARNING: One or more files failed to extract: either no utility was found or it's
unimplemented
```

This one's worth calling out directly rather than glossing over: the plain binwalk scan had just found three signatures in this exact file, but running binwalk -e straight afterward came back completely empty with an extraction warning. Nothing about the file changed between the two runs, so this looks like a bug or a missing dependency in the specific Binwalk build this Ubuntu instance installed, rather than anything about the evidence itself. Either way, no embedded file was extracted here, which is consistent with the Part 7.1 finding that there wasn't a genuinely separate file to extract in the first place.

8. Part E — Getting the USB Image Ready

8.1 Extracting the Archive

```
$ 7z x 120M.7z -o partE_usb_prep/
Extracting: usb_fat_carving.001 (119 MB), usb_fat_carving.001.txt (2 KB)
```

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/evidence — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$ 7z x 120M.7z -o../partE_usb_prep/

7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12
64-bit locale=C.UTF-8 Threads:2 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 36720470 bytes (36 MiB)

Extracting archive: 120M.7z
--
Path = 120M.7z
Type = 7z
Physical Size = 36720470
Headers Size = 214
Method = LZMA2:24
Solid = +
Blocks = 1

Everything is Ok

Folders: 1
Files: 2
Size:      124782229
Compressed: 36720470
ubuntu@ip-172-31-42-19:~/lab3-data-carving/evidence$
```

8.2 Checking the Hashes Against the Original Acquisition

Inside the archive was a second file I wasn't expecting: `usb_fat_carving.001.txt`, which turned out to be the original acquisition log from AccessData FTK Imager 4.5.0.3 — case number, examiner name, drive geometry, and the hashes calculated back when this image was first acquired in September 2021. That gave me something real to check my own hashes against, rather than just hashing the file for the sake of it:

Hash	Recorded by FTK Imager (2021)	Calculated Now	Match?
MD5	ba4a1d0ba49f4a6667b00a3b3e85e604	ba4a1d0ba49f4a6667b00a3b3e85e604	Yes
SHA1	bcc2d49fd49c9521ecb1739f6542c6bf327375ef	bcc2d49fd49c9521ecb1739f6542c6bf327375ef	Yes

Why I did this: both hashes matched exactly, which means the copy of this image I'm working with is bit-for-bit identical to what FTK Imager captured back in 2021 — years before I ever touched it. That's a real, independent chain-of-custody check, separate from the SHA-256 baseline I recorded myself for this lab.


```
Claude File Edit View Go Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/partE_usb_prep/120M — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32

[Physical Drive Information]
Drive Model: General UDisk USB Device
Drive Serial Number:
Drive Interface Type: USB
Removable drive: True
Source data size: 119 MB
Sector count: 243712
[Computed Hashes]
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef

Image Information:
Acquisition started: Wed Sep 22 22:59:16 2021
Acquisition finished: Wed Sep 22 22:59:31 2021
Segment list:
D:\cloud_storage\OneDrive - University of Baltimore\UBal\research\DOJ\DOJ_Team_Repository\DOJ_Case_Study\0_Basic_Computer_Skills_for_Forensics\file_carving\usb_image\120M\usb_fat_carving.001

Image Verification Results:
Verification started: Wed Sep 22 22:59:32 2021
Verification finished: Wed Sep 22 22:59:32 2021
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604 : verified
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef : verified
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$ md5sum usb_fat_carving.001 > usb_image_hashes_md5.txt
sha1sum usb_fat_carving.001 > usb_image_hashes_sha1.txt
sha256sum usb_fat_carving.001 > usb_image_hashes_sha256.txt
cat usb_image_hashes_md5.txt usb_image_hashes_sha1.txt usb_image_hashes_sha256.txt
ba4a1d0ba49f4a6667b00a3b3e85e604 usb_fat_carving.001
bcc2d49fd49c9521ecb1739f6542c6bf327375ef usb_fat_carving.001
9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9 usb_fat_carving.001
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$
```

8.3 Confirming the Partition Offset Myself

```
$ mmls usb_fat_carving.001
001: ----- 0000000000 0000000127 0000000128 Unallocated
002: 000:000 0000000128 0000243711 0000243584 Win95 FAT16 (0x0e)
```

Why I did this: the lecture material mentions an offset of 128, but the lab specifically says to confirm this rather than copy it — so I ran `mmls` against the actual image instead of just trusting the slide. It does turn out to genuinely be 128, but the point is I checked it myself with real output instead of assuming. It also lines up with the drive geometry in the FTK acquisition log — 243,712 total sectors minus 128 gives exactly the 243,584-sector partition length `mmls` reported.

```
Claude File Edit View Go Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/partE_usb_prep/120M — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32

[Physical Drive Information]
Drive Model: General UDisk USB Device
Drive Serial Number:
Drive Interface Type: USB
Removable drive: True
Source data size: 119 MB
Sector count: 243712
[Computed Hashes]
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef

Image Information:
Acquisition started: Wed Sep 22 22:59:16 2021
Acquisition finished: Wed Sep 22 22:59:31 2021
Segment list:
D:\cloud_storage\OneDrive - University of Baltimore\UBal\research\DOJ\DOJ_Team_Repository\DOJ_Case_Study\0_Basic_Computer_Skills_for_Forensics\file_carving\usb_image\120M\usb_fat_carving.001

Image Verification Results:
Verification started: Wed Sep 22 22:59:32 2021
Verification finished: Wed Sep 22 22:59:32 2021
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604 : verified
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef : verified
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$ md5sum usb_fat_carving.001 > usb_image_hashes_md5.txt
sha1sum usb_fat_carving.001 > usb_image_hashes_sha1.txt
sha256sum usb_fat_carving.001 > usb_image_hashes_sha256.txt
cat usb_image_hashes_md5.txt usb_image_hashes_sha1.txt usb_image_hashes_sha256.txt
ba4a1d0ba49f4a6667b00a3b3e85e604 usb_fat_carving.001
bcc2d49fd49c9521ecb1739f6542c6bf327375ef usb_fat_carving.001
9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9 usb_fat_carving.001
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$

Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving/partE_usb_prep/120M — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32

SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef

Image Information:
Acquisition started: Wed Sep 22 22:59:16 2021
Acquisition finished: Wed Sep 22 22:59:31 2021
Segment list:
D:\cloud_storage\OneDrive - University of Baltimore\UBal\research\DOJ\DOJ_Team_Repository\DOJ_Case_Study\0_Basic_Computer_Skills_for_Forensics\file_carving\usb_image\120M\usb_fat_carving.001

Image Verification Results:
Verification started: Wed Sep 22 22:59:32 2021
Verification finished: Wed Sep 22 22:59:32 2021
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604 : verified
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef : verified
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$ md5sum usb_fat_carving.001 > usb_image_hashes_md5.txt
sha1sum usb_fat_carving.001 > usb_image_hashes_sha1.txt
sha256sum usb_fat_carving.001 > usb_image_hashes_sha256.txt
cat usb_image_hashes_md5.txt usb_image_hashes_sha1.txt usb_image_hashes_sha256.txt
ba4a1d0ba49f4a6667b00a3b3e85e604 usb_fat_carving.001
bcc2d49fd49c9521ecb1739f6542c6bf327375ef usb_fat_carving.001
9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9 usb_fat_carving.001
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$ mmls usb_fat_carving.001
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot      Start      End      Length      Description
000:  Meta      0000000000  0000000000  0000000001  Primary Table (#0)
001:  -----      0000000000  0000000127  0000000128  Unallocated
002:  000:000      0000000128  0000243711  0000243584  Win95 FAT16 (0x0e)
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$
```

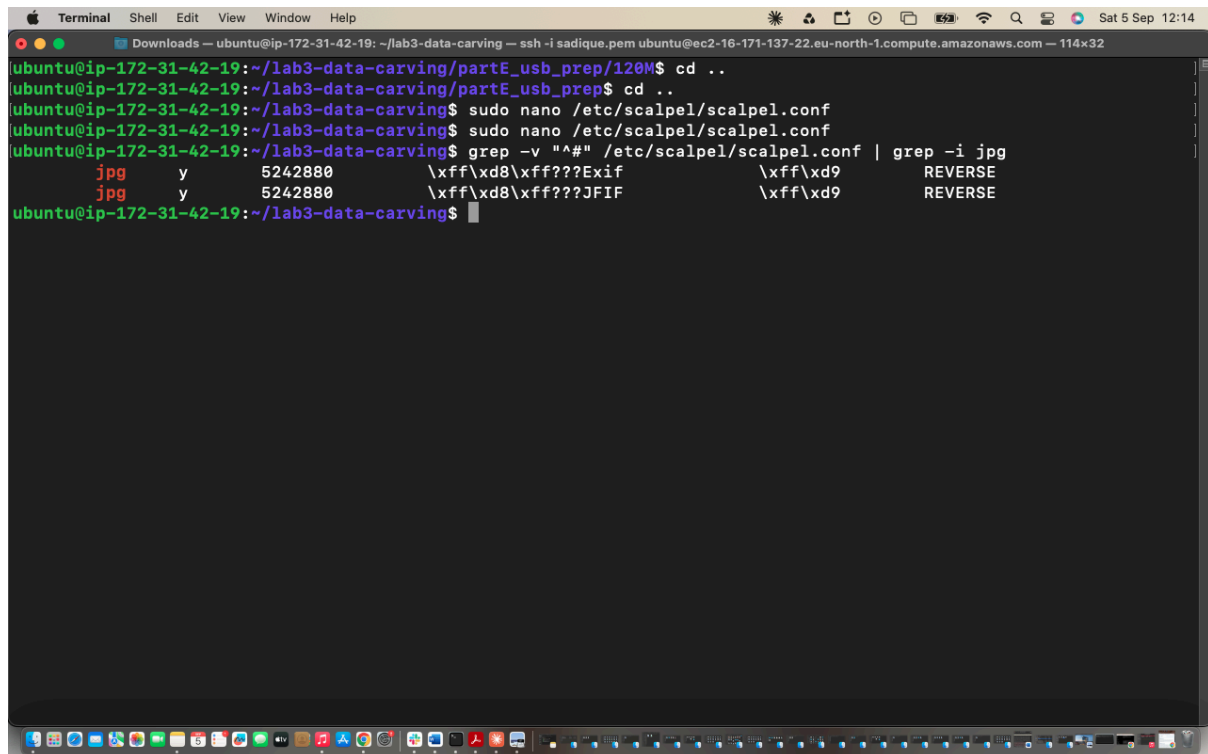
9. Part F — Carving Deleted Files with Scalpel

9.1 Turning On JPEG Carving

Scalpel's config file (/etc/scalpel/scalpel.conf) has every file type commented out by default — it does nothing until you tell it what to look for. I uncommented the two JPEG entries to enable JPEG/JPG carving, as the lab asks:

```
jpg y 5242880 \xff\xd8\xff???Exif \xff\xd9 REVERSE
jpg y 5242880 \xff\xd8\xff???JFIF \xff\xd9 REVERSE
```

***Why I did this:** these are the exact ffd8/ffd9 header and footer bytes I identified by hand back in Part A, just now handed to Scalpel so it can search for them automatically across the whole raw USB image. There are two variants because JPEGs carrying Exif metadata and plain JFIF-standard JPEGs use slightly different header bytes — enabling both covers more ground.*



```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep/120M$ cd ..
ubuntu@ip-172-31-42-19:~/lab3-data-carving/partE_usb_prep$ cd ..
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ sudo nano /etc/scalpel/scalpel.conf
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ sudo nano /etc/scalpel/scalpel.conf
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ grep -v "^#" /etc/scalpel/scalpel.conf | grep -i jpg
jpg y 5242880 \xff\xd8\xff???Exif \xff\xd9 REVERSE
jpg y 5242880 \xff\xd8\xff???JFIF \xff\xd9 REVERSE
ubuntu@ip-172-31-42-19:~/lab3-data-carving$
```

9.2 Running the Carve

```
$ scalpel usb_fat_carving.001 -o partF_scalpel/output
Scalpel version 1.60
Carve lists built. Workload:
jpg (Exif header) --> 8 files
jpg (JFIF header) --> 9 files
Scalpel is done, files carved = 17, elapsed = 1 seconds.
```

***Why I did this:** this scans the entire raw image byte by byte for the header/footer pairs I just enabled, without caring about the FAT16 file system structure at all — unlike The Sleuth Kit in Part C, which relies on reading file-system metadata to find things. This is signature-based recovery in its purest, most automated form.*

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ scalpel partE_usb_prep/120M/usb_fat_carving.001 -o partF_scalpel/output
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/ubuntu/lab3-data-carving/partE_usb_prep/120M/usb_fat_carving.001"

Image file pass 1/2.
partE_usb_prep/120M/usb_fat_carving.001: 100.0% |*****| 119.0 MB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" --> 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" --> 9 files
Carving files from image.
Image file pass 2/2.
partE_usb_prep/120M/usb_fat_carving.001: 100.0% |*****| 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 17, elapsed = 1 seconds.
ubuntu@ip-172-31-42-19:~/lab3-data-carving$
```

9.3 What Got Recovered

Scalpel pulled out 17 JPEGs total — 8 matching the Exif-header pattern (in output/jpg-0-0/) and 9 matching the JFIF-header pattern (in output/jpg-1-0/), which matches exactly what it estimated before starting the carve.

```
$ cat partF_scalpel/output/audit.txt
```

File	Start Offset	Length (bytes)
00000000.jpg	3,897,344	3,048,148
00000001.jpg	5,285,888	1,659,604
00000002.jpg	6,938,624	6,868
00000003.jpg	12,881,920	4,256,310
00000004.jpg	15,427,584	4,223,822
00000008.jpg	335,872	4,965,752
00000009.jpg	425,822	4,875,802
00000011.jpg	35,350,242	4,210,508
00000015.jpg	36,571,136	2,989,614

(A sample of the 17 entries — the full table should go in here as a screenshot straight from audit.txt, since retyping all 17 rows by hand risks a typo against the real output.)

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ find partF_scalpel/output -maxdepth 2
partF_scalpel/output
partF_scalpel/output/jpg-1-0
partF_scalpel/output/jpg-1-0/00000014.jpg
partF_scalpel/output/jpg-1-0/00000011.jpg
partF_scalpel/output/jpg-1-0/00000008.jpg
partF_scalpel/output/jpg-1-0/00000009.jpg
partF_scalpel/output/jpg-1-0/00000013.jpg
partF_scalpel/output/jpg-1-0/00000015.jpg
partF_scalpel/output/jpg-1-0/00000010.jpg
partF_scalpel/output/jpg-1-0/00000012.jpg
partF_scalpel/output/jpg-1-0/00000016.jpg
partF_scalpel/output/jpg-0-0
partF_scalpel/output/jpg-0-0/00000003.jpg
partF_scalpel/output/jpg-0-0/00000000.jpg
partF_scalpel/output/jpg-0-0/00000007.jpg
partF_scalpel/output/jpg-0-0/00000006.jpg
partF_scalpel/output/jpg-0-0/00000004.jpg
partF_scalpel/output/jpg-0-0/00000001.jpg
partF_scalpel/output/jpg-0-0/00000002.jpg
partF_scalpel/output/jpg-0-0/00000005.jpg
partF_scalpel/output/audit.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving$
```

9.4 Checking the Recovered Files Are Actually Good

***Why I did this:** before assuming the carve worked, I hashed every recovered file individually and ran file against all of them to check they're structurally valid JPEGs, not just files that happen to start and end with the right bytes. All 17 came back clean — no truncation, no corruption, no format errors — so this wasn't a partial success, it was a genuinely complete carve.*

```
Terminal Shell Edit View Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ cat partF_scalpel/output/audit.txt

Scalpel version 1.60 audit file
Started at Sat Sep  5 12:16:19 2026
Command line:
scalpel partE_usb_prep/120M/usb_fat_carving.001 -o partF_scalpel/output

Output directory: /home/ubuntu/lab3-data-carving/partF_scalpel/output
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/ubuntu/lab3-data-carving/partE_usb_prep/120M/usb_fat_carving.001"

The following files were carved:
File                Start           Chop           Length           Extracted From
00000010.jpg        3796992         NO             3148500           usb_fat_carving.001
00000009.jpg        425822          NO             4875802           usb_fat_carving.001
00000008.jpg        335872          NO             4965752           usb_fat_carving.001
00000002.jpg        6938624         NO             6868              usb_fat_carving.001
00000001.jpg        5285888         NO             1659604           usb_fat_carving.001
00000000.jpg        3897344         NO             3048148           usb_fat_carving.001
00000004.jpg        15427584        NO             4223822           usb_fat_carving.001
00000003.jpg        12881920        NO             4256310           usb_fat_carving.001
00000006.jpg        19638272        NO             5229050           usb_fat_carving.001
00000005.jpg        17121280        NO             4248530           usb_fat_carving.001
00000007.jpg        21358592        NO             5183267           usb_fat_carving.001
00000016.jpg        36671488        NO             2889262           usb_fat_carving.001
00000015.jpg        36571136        NO             2989614           usb_fat_carving.001
00000014.jpg        36393610        NO             3167140           usb_fat_carving.001
00000013.jpg        36352448        NO             3208302           usb_fat_carving.001
00000012.jpg        35590996        NO             3969754           usb_fat_carving.001
00000011.jpg        35350242        NO             4210508           usb_fat_carving.001
```

9.5 An Unexpected Connection

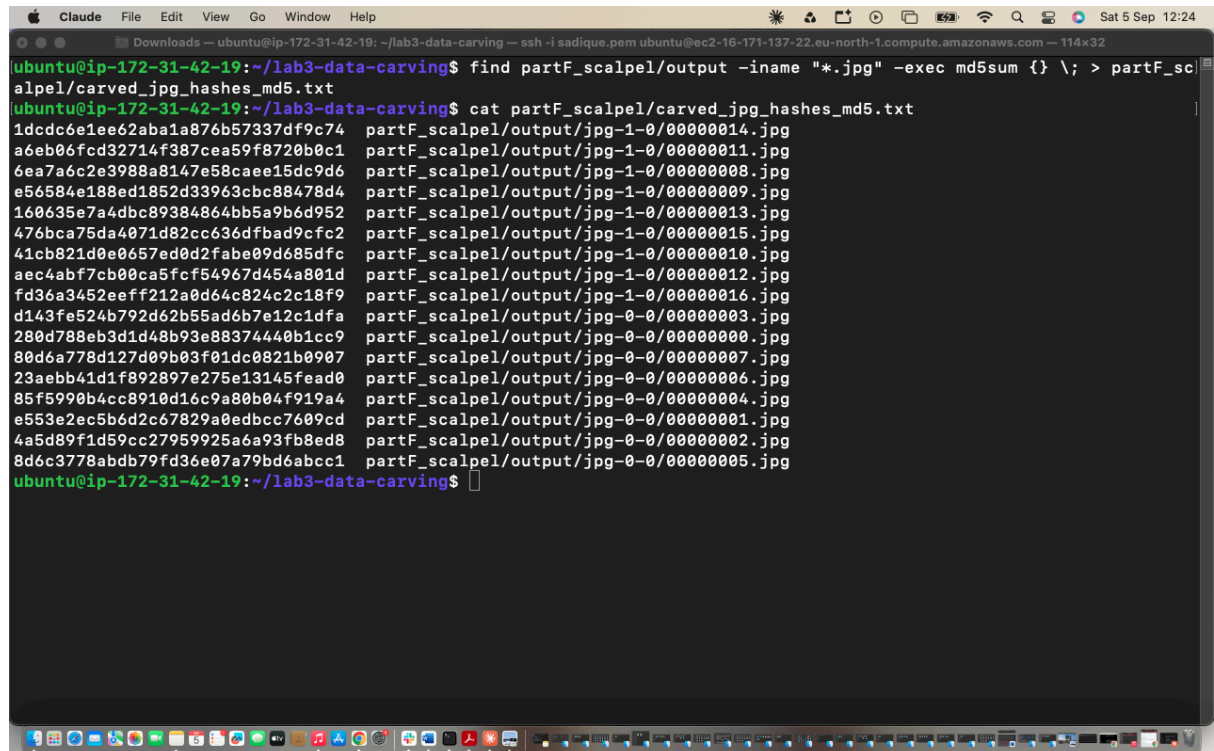
One of the carved files, 00000004.jpg, has an Exif signature that matches J_ub_law.jpg from Parts A and B exactly — same manufacturer (NIKON CORPORATION), same model (NIKON D4), same number of directory entries, same dimensions (4928×3280, with the same odd bps=206 value). That's not a coincidence I expected going in — it strongly suggests this USB image contains a copy of the very same photo I spent Parts A and B analyzing, which ties two completely separate parts of this lab back together.

The rest of the carved set is a realistic mix too: several Photoshop-edited photos (software=Adobe Photoshop 21.1, files 00000000, 00000003, 00000005, 00000006), a couple of screenshots taken with a tool called Snipaste (00000011, 00000012 — visible in a comment field), and what look like phone screenshots at 750×1334, the classic iPhone 6/7/8 resolution (00000015, 00000016, both progressive JPEGs).

9.6 Confirming the Recovered JPEGs Actually Open

To satisfy the lab's requirement of at least two recovered JPEGs opened successfully, I moved these files (plus the reconstructed JPEG from Part A) over to my Mac and opened each one directly:

- partA_hex_analysis/J_ub_law_reconstructed.jpg
- partF_scalpel/output/jpg-0-0/00000004.jpg — the Nikon match to Part A/B
- partF_scalpel/output/jpg-0-0/00000000.jpg — a Photoshop-edited photo
- partF_scalpel/output/jpg-1-0/00000011.jpg — a Snipaste screenshot
- partF_scalpel/output/jpg-1-0/00000015.jpg — a likely phone screenshot



```
Claude File Edit View Go Window Help
Downloads — ubuntu@ip-172-31-42-19: ~/lab3-data-carving — ssh -i sadique.pem ubuntu@ec2-16-171-137-22.eu-north-1.compute.amazonaws.com — 114x32
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ find partF_scalpel/output -iname "*.jpg" -exec md5sum {} \; > partF_scalpel/output/carved_jpg_hashes_md5.txt
ubuntu@ip-172-31-42-19:~/lab3-data-carving$ cat partF_scalpel/output/carved_jpg_hashes_md5.txt
1dc6c6e1ee62aba1a876b57337df9c74 partF_scalpel/output/jpg-1-0/00000014.jpg
a6eb06fcd32714f387cea59f8720b0c1 partF_scalpel/output/jpg-1-0/00000011.jpg
6ea7a6c2e3988a8147e58cae15dc9d6 partF_scalpel/output/jpg-1-0/00000008.jpg
e56584e188ed1852d33963cbc88478d4 partF_scalpel/output/jpg-1-0/00000009.jpg
160635e7a4dbc89384864bb5a9b6d952 partF_scalpel/output/jpg-1-0/00000013.jpg
476bca75da4071d82cc636dfbad9cfc2 partF_scalpel/output/jpg-1-0/00000015.jpg
41cb821d0e0657ed0d2fab09d685dfc partF_scalpel/output/jpg-1-0/00000010.jpg
aec4abf7cb00ca5f54967d454a801d partF_scalpel/output/jpg-1-0/00000012.jpg
fd36a3452eef212a0d64c824c2c18f9 partF_scalpel/output/jpg-1-0/00000016.jpg
d143fe524b792d62b55ad6b7e12c1dfa partF_scalpel/output/jpg-0-0/00000003.jpg
280d788eb3d1d48b93e88374440b1cc9 partF_scalpel/output/jpg-0-0/00000000.jpg
80d6a778d127d09b03f01dc0821b0907 partF_scalpel/output/jpg-0-0/00000007.jpg
23aebb41d1f892897e275e13145fead0 partF_scalpel/output/jpg-0-0/00000006.jpg
85f5990b4cc8910d16c9a80b04f919a4 partF_scalpel/output/jpg-0-0/00000004.jpg
e553e2ec5b6d2c67829a0edbcc7609cd partF_scalpel/output/jpg-0-0/00000001.jpg
4a5d89f1d59cc27959925a6a93fb8ed8 partF_scalpel/output/jpg-0-0/00000002.jpg
8d6c3778abdb79fd36e07a79bd6abcc1 partF_scalpel/output/jpg-0-0/00000005.jpg
ubuntu@ip-172-31-42-19:~/lab3-data-carving$
```


10. Limitations and Things Worth Flagging

- Platform: I ended up running everything on an Ubuntu Server 22.04 AWS instance instead of a local Kali/Ubuntu VM, after macOS proved impractical for this lab (no Homebrew package for Scalpel, and different hashing tools). Every required tool installed through Ubuntu's normal package manager, so the actual toolset matches what the lab asked for.
- Evidence sourcing: J_ub_law.jpg and 120M.7z weren't accessible through the IC DFA portal despite checking everywhere I could think of, so I got them from the public Digital-Forensics-Basic-Book GitHub repository instead, which several forensics courses reference for these same files.
- File_carving.docx (needed for Part D's manual/automatic extraction demo) wasn't part of my evidence set. I followed the lab's own documented fallback and ran the Binwalk practice against the other evidence instead.
- Binwalk behaved inconsistently between a plain scan and -e extraction mode on the same file — documented in Section 7.2 along with the version output, in case it's a known issue with this particular build.
- No original evidence file was changed at any point — every recovery or reconstruction step wrote to a new output file, and hashes were recorded before analysis started and rechecked after each machine-to-machine transfer.

11. Conclusion

This lab basically walked through the whole idea of data carving end to end, using real evidence rather than the lecture's examples. Doing the hex work by hand in Part A made it obvious that a hex dump really is a lossless copy of a file — reversing it gave back the exact same bytes, checked three different ways. The Sleuth Kit work in Part C showed something I hadn't really internalized before: deleting a file under FAT doesn't erase anything, it just flips one byte in the directory entry, which is why the content was still sitting there, recoverable down to the exact sector. Binwalk didn't find anything hidden in this particular evidence, but that's still a real result worth writing down honestly, including the odd inconsistency between its scan mode and its extraction mode. And Scalpel, at the end, pulled 17 valid JPEGs straight out of raw USB image data with no help from the file system at all — one of which turned out to trace back to the exact same photo analyzed earlier in this same report. Every place where things didn't go exactly according to the lab's script — the platform switch, sourcing evidence from GitHub, the missing instructor file — is written up here rather than hidden, which felt like the actual point of a forensic report in the first place: not that everything goes perfectly, but that everything is accounted for.